

这里是计算几何

- 使用建议

- 精度：定义dcmp来判断所有double
- 向量化
- 调试：`cout << fixed << setprecision(10);`

前置头

```
const double EPS = 1e-9;
const double PI = acos(-1.0);

inline int dcmp(double x) {
    if (x < -EPS) return -1;
    if (x > EPS) return 1;
    return 0;
} // 这个函数可以做到：1, 提取数字的符号 2, 比较两个数的大小: 调用的时候: dcmp(a-b)
inline double sqr(double x) { return x * x; } // 取平方
```

struct 点(包含重载运算符)

```
struct Point {
    double x, y; // 点

    Point(double x=0, double y=0):x(x),y(y){}
    /*
    支持高效创建点
    例如 Point p2(3, 4);    // (3,4)
    */

    Point operator+(const Point &b) const { return Point(x+b.x, y+b.y); }
    Point operator-(const Point &b) const { return Point(x-b.x, y-b.y); }
    Point operator*(double p) const { return Point(x*p, y*p); }
    Point operator/(double p) const { return Point(x/p, y/p); }

    /*
    重载运算符, 进行点的向量运算
    例如: Point a(1,2), b(3,4);
           Point c = a + b;    // c = (4,6)
    重载乘除: 对坐标进行放缩
           Point v = p * 2.0;   // 坐标都 ×2
           Point unit = v / v.len(); // 单位向量 (后面会讲)
    */

    double operator^(const Point &b) const { return x*b.y - y*b.x; } // 定义 ^ 为
    叉积(后面可能会用到) 相当于后文的cross函数
    bool operator==(const Point &b) const { return dcmp(x-b.x)==0 && dcmp(y-
    b.y)==0; }
    /*
```

```

判断两个点在误差内是否相等（重合）
*/
double len() const { return hypot(x,y); } // 长度
double len2() const { return x*x + y*y; } // 平方
//调用方式: double dist = (p1 - p2).len();

Point unit() const { return *this / len(); } // 单位向量
Point perp() const { return Point(-y, x); } // 逆时针90°
/*
调用方式: Point v(3, 4);
           Point u = v.unit(); // u 是v的单位向量
           Point p = v.perp(); // p = (-4, 3)
*/
double dot(const Point &b) const { return x*b.x + y*b.y; } //点积
double cross(const Point &b) const { return x*b.y - y*b.x; } //叉积
//叉积数学公式 $a \times b = x_1 y_2 - x_2 y_1 = |a| |b| \sin \theta$ 
double angle() const { return atan2(y,x); } // 与x轴夹角

Point rotate(double theta) const { // 绕原点逆时针旋转, theta填弧度, 填
的是几分之几的Pi
    double c=cos(theta), s=sin(theta);
    return Point(x*c - y*s, x*s + y*c);
}
};
using Vec = Point; //不要和vector撞了

```

常用函数

```

// 向量叉积 (a×b)
double Cross(Point a, Point b) { return a.x*b.y - a.y*b.x; }

// 向量点积
double Dot(Point a, Point b) { return a.x*b.x + a.y*b.y; }

// 两点距离
double Dist(Point a, Point b) { return (a-b).len(); } //需要重载.len()

// 点到直线距离 (p 到 ab)
double DistToLine(Point p, Point a, Point b) {
    return fabs(Cross(p-a, b-a)) / Dist(a,b);
}
//两点之间距离 (简单版)
double disPointPoint(Point u, Point v){
    return sqrt((u.x-v.x)*(u.x-v.x)+(u.y-v.y)*(u.y-v.y));
}

// 点到线段距离
double DistToSeg(Point p, Point a, Point b) {
    if (a==b) return Dist(p,a);
    Vec v1 = b-a, v2 = p-a, v3 = p-b;
    if (dcmp(Dot(v1,v2)) < 0) return Dist(p,a);

```

```

    if (dcmp(Dot(v1,v3)) > 0) return Dist(p,b);
    return DistToLine(p,a,b);
}

// 点 p 在线段 ab 上的投影
Point Proj(Point p, Point a, Point b) {
    Vec v = b-a;
    return a + v * (Dot(v, p-a) / v.len2());
}

// 判两线段AB和CD是否相交 (含端点)
bool SegIntersect(Point a,Point b,Point c,Point d) {
    auto ccw = [](Point p,Point q,Point r)
    { return dcmp(Cross(q-p, r-p)); };//作用: 判断点 r 在直线 pq 的哪一侧
    return ccw(a,b,c)*ccw(a,b,d)<=0 && ccw(c,d,a)*ccw(c,d,b)<=0;
}

/*原理: 两条线段相交的充要条件:
C 和 D 在 AB 的两侧或线上
A 和 B 在 CD 的两侧或线上*/

// 判断点 p 是否在 线段 ab 上 (包括端点 a 和 b)
bool OnSeg(Point p, Point a, Point b) {
    return dcmp(Cross(p-a,b-a))==0 && dcmp(Dot(p-a,p-b))<=0;
}

```

- 共线: $\text{Cross}(p-a, b-a) = (p-a) \times (b-a)$
- 在点区间内: $\text{Dot}(p-a, p-b) \leq 0$

排序(大部分时间我手搓lambda, 这是属于提醒自己)

```

//以x升序排列, 其次以y升序排列
bool sortXupYup(Point u,Point v){
    if(u.x!=v.x) return u.x<v.x;
    else return u.y<v.y;
}

//以y升序排列, 其次以x升序排列
bool sortYupXup(Point u,Point v){
    if(u.y!=v.y) return u.y<v.y;
    else return u.x<v.x;
}

//对点进行极角排序
bool sortPointAngle(Point a,Point b){
    if(a.quad()!=b.quad()) return a.quad()<b.quad();
    return (a^b)>0;
}

```

几何公式

```

//余弦定理 ab为邻边, c为对边
double cosLaw(double a,double b,double c){
    return (a*a+b*b-c*c)/(2*a*b);
}

//三角形面积叉积法
double mianj(Point u,Point v,Point w){
    return abs((v-u)^(w-u))/2.0;
}

//求多边形面积 (只能求凸包)
double polygonArea(Point *u,int size){
    double area=0;
    Point begin=u[1];
    /*
    由第一个点起始的顺序叉积,
    面积值为边之间连线的封闭部分, 叉积能够计算容斥部分
    */
    for(int i=3;i<=size;i++) area+=((u[i-1]-begin)^(u[i]-begin))/2;
    return area;
}

// 适用于任意简单多边形的计算面积公式, 注意顶点要按照顺序给出
double polygonArea2(Point *u, int size){
    if (size < 3) return 0.0; // 点/线没有面积
    double area = 0;
    u[size+1] = u[1]; // 哨兵, 使循环闭合
    for (int i = 1; i <= size; i++) {
        // 假设你已经重载了 ^
        // area += (u[i] ^ u[i+1]);
        area += u[i].cross(u[i+1]);
    }
    return fabs(area / 2.0);
}

```

正多边形&&圆

```

// 正 n 边形外接圆半径 R → 边长
double RegSide(double R, int n) { return 2 * R * sin(PI / n); }

// 正 n 边形内接圆半径 r → 边长
double RegSideIn(double r, int n) { return 2 * r * tan(PI / n); }

// 用于生成一个正多边形的顶点坐标。(中心 O, 半径 R, 起始角度 theta0)
vector<Point> RegularPoly(Point O, double R, int n, double theta0 = 0) {
    vector<Point> res;
    double ang = 2*PI/n;
    for (int i = 0; i < n; ++i) {
        double a = theta0 + i*ang;
        res.emplace_back(O.x + R*cos(a), O.y + R*sin(a));
    }
}

```

```

    }
    return res;
}

```

```

// 圆的面积 - 两点 + 半径 构造圆
struct Circle {
    Point c; // 圆心
    double r; // 半径

    Circle(Point c=Point(), double r=0):c(c),r(r){}
    // 调用: Circle cir(Point(1, 1), 5.0); // 圆心 (1,1), 半径 5

    // 三点确定圆 (非共线)
    // 调用: Circle cir1(A, B, C);
    Circle(Point a, Point b, Point c) {
        Point ab = (a+b)*0.5, bc = (b+c)*0.5;
        Vector d1 = (b-a).perp(), d2 = (c-b).perp();
        Point o = LineIntersect(ab, ab+d1, bc, bc+d2);
        *this = Circle(o, Dist(o,a));
    }
};

```

- 三点定圆需要定义LineIntersect函数 (详见后面)
- 生成正多边形顶点坐标几何原理: 使用极坐标转直角坐标公式:

$$x = O_x + R \cdot \cos(\theta), \quad y = O_y + R \cdot \sin(\theta)$$

从 θ_0 开始, 每次旋转 $2\pi/n$ 弧度, 生成一个顶点。

- 用法示例

```

Point center = {0, 0}; auto pentagon = RegularPoly(center, 1.0, 5, 0);
// 正五边形, 半径1, 顶点从x轴正方向开始 会生成 5 个点, 均匀分布在单位圆上。

```

直线&线段交点

需要Point重载+ - *; 需要using Vec = Point; 需要点积叉积; 判断p是否在AB上; 需要标准线段相交判断(不一定)

```

// 直线参数方程交点
Point LineIntersect(Point p1, Vec v1, Point p2, Vec v2) {
    double t = Cross(p2-p1, v2) / Cross(v1, v2);
    return p1 + v1 * t;
}

// 两线段交点 (若相交返回 true, 交点写入 its)
bool SegIntersection(Point a, Point b, Point c, Point d, Point &its) {

```

```

    if (!SegIntersect(a,b,c,d)) return false;
    double t = Cross(c-a, d-c) / Cross(b-a, d-c);
    its = Point(a.x + t*(b.x-a.x), a.y + t*(b.y-a.y));
    return true;
}

```

- 求线段交点核心代码片段: `double t = Cross(c-a, d-c) / Cross(b-a, d-c); its = Point(a.x + t*(b.x-a.x), a.y + t*(b.y-a.y));`

凸包：我不会

```

int ConvexHull(vector<Point> &p, vector<Point> &hull) {
    int n = p.size(); if (n<=1) { hull=p; return n; }
    sort(p.begin(), p.end(), [](Point a, Point b){
        return dcmp(a.x-b.x)<0 || (dcmp(a.x-b.x)==0 && dcmp(a.y-b.y)<0);
    });
    hull.clear(); hull.reserve(n);
    // 下凸包
    for (int i=0;i<n;i++){
        while (hull.size()>=2 && dcmp(Cross(hull.back()-hull[hull.size()-2], p[i]-hull.back())) <=0)
            hull.pop_back();
        hull.push_back(p[i]);
    }
    int t = hull.size();
    // 上凸包
    for (int i=n-2;i>=0;i--){
        while (hull.size()>t && dcmp(Cross(hull.back()-hull[hull.size()-2], p[i]-hull.back())) <=0)
            hull.pop_back();
        hull.push_back(p[i]);
    }
    hull.pop_back(); // 去掉重复起点
    return hull.size();
}

```

旋转卡壳 – 直径、对踵(我不会)

```

// 凸多边形直径（最远点对）
double Diameter(const vector<Point> &hull) {
    int n = hull.size(); if (n<=1) return 0;
    if (n==2) return Dist(hull[0],hull[1]);
    int i=0,j=1;
    for (int k=0;k<n;k++){
        if (dcmp(hull[k].x - hull[i].x)>0) i=k;
        j = (i+1)%n;
    }
    int si=i,sj=j;
    double res = 0;
    while (i!=sj || j!=si){

```

```

        res = max(res, Dist(hull[i],hull[j]));
        if (dcmp(Cross(hull[(i+1)%n]-hull[i], hull[(j+1)%n]-hull[j])) < 0) i=
(i+1)%n;
        else j=(j+1)%n;
    }
    return res;
}

```

最小圆覆盖 $O(n)$

```

Circle MinCircleCover(vector<Point> p) {
    random_shuffle(p.begin(), p.end());
    int n = p.size();
    Circle c(p[0], 0);
    for (int i=1;i<n;i++) if (dcmp(Dist(c.c, p[i]) - c.r) > 0) {
        c = Circle(p[i],0);
        for (int j=0;j<i;j++) if (dcmp(Dist(c.c, p[j]) - c.r) > 0) {
            c = Circle((p[i]+p[j])*0.5, Dist(p[i],p[j])*0.5);
            for (int k=0;k<j;k++) if (dcmp(Dist(c.c, p[k]) - c.r) > 0) {
                c = Circle(p[i], p[j], p[k]);
            }
        }
    }
    return c;
}

```

半平面交 $O(n \log n)$

```

struct Halfplane {
    Point p; Vector v; double ang;
    Halfplane(){}
    Halfplane(Point a, Point b){ p=a; v=b-a; ang=atan2(v.y,v.x); }
    bool operator<(const Halfplane &b) const { return ang < b.ang; }
    Point intersect(const Halfplane &b) const {
        double t = Cross(b.p-p, b.v) / Cross(v, b.v);
        return p + v*t;
    }
};

vector<Point> HalfplaneIntersect(vector<Halfplane> hp) {
    sort(hp.begin(), hp.end());
    int n = hp.size(), l=0, r=0;
    vector<Halfplane> q(n+5);
    vector<Point> res;
    q[r++] = hp[0];
    for (int i=1;i<n;i++){
        while (l+1 < r && dcmp(Cross(hp[i].p - q[r-2].intersect(q[r-1]), hp[i].v))
< 0) --r;
        while (l < r && dcmp(Cross(hp[i].p - q[l].intersect(q[l+1]), hp[i].v)) <
0) ++l;
    }
}

```

```

        q[r++] = hp[i];
    }
    while (l+1 < r && dcmp(Cross(q[l].p - q[r-2].intersect(q[r-1]), q[l].v)) < 0)
--r;
    while (l < r && dcmp(Cross(q[r-1].p - q[l].intersect(q[l+1]), q[r-1].v)) < 0)
++l;
    if (r-l<=1) return res;
    for (int i=l;i<r;i++) res.push_back(q[i].intersect(q[(i+1==r)?l:i+1]));
    return res;
}

```

分治

来自知乎复制粘贴[链接](#) 感谢大佬整理

```

//分治
void Nearestpoint(point *a,int l,int r,int &u,int &v,double &d){
/*
    应当注意距离的定义形式，如果为平方的形式，
    则代码中绝对值的部分也应当改为平方
*/
    if(l==r) return ;
    if(l+1==r) {
        if(disPointPoint(a[l],a[r])<d) d=disPointPoint(a[l],a[r]),u=l,v=r;
        return ;
    }
    int mid=l+r>>1,m=0;
    Nearestpoint(a,l,mid,u,v,d),Nearestpoint(a,mid+1,r,u,v,d);
    point b[r-l+10];
    for(int i=l;i<=r;i++){
        if(abs(a[i].x-a[mid].x)<d) b[++m]=a[i];
    }
    sort(b+1,b+1+m,sortYupXup);
    for(int i=1;i<=m;i++){
        for(int j=i+1;j<=m&&abs(b[i].y-b[j].y)<d;j++){
            if(d>disPointPoint(b[i],b[j]))d=disPointPoint(b[i],b[j]),u=i,v=j;
        }
    }
    return ;
}
//分治法的入口,分治前需要排序
double mindisset1(point* a,int size,int &u,int &v){
    //函数起始入口
    double minn=1e18;
    sort(a+1,a+1+size,sortXupYup);
    Nearestpoint(a,1,size,u,v,minn);
    return minn;
}
//multiset
double mindisset2(point* u,int size){
    double ans=1e20;

```



```
struct cmp_y {
    bool operator()(const point &a, const point &b) const { return a.y < b.y; }
};
multiset<point,cmp_y>s;
for (int i = 1, l = 1; i <=size; i++) {
    while (l < i && u[i].x - u[l].x >= ans) s.erase(s.find(u[l++]));
    for (auto it = s.lower_bound(point{u[i].x, u[i].y - ans});
        it != s.end() && it->y - u[i].y < ans; it++){
        if(disPointPoint(*it,u[i])<ans) ans=disPointPoint(*it,u[i]);
    }
    s.insert(u[i]);
}
return ans;
}
```